AITA-INTELLIGENT — Software Requirements Specification (SRS)

Version: 1.0 **Date Created:** 09/07/2026 **Development Team:** SWP391 – Group 3 **Semester:** Fall 2026 – FPT University

Table of Contents

1. Introduction

1.1. Document Purpose

This Software Requirements Specification (SRS) document describes all functional and non-functional requirements of the **AITA-Intelligent** system (AI-Powered Teaching Assistant & AST Code Analytics Platform). This document serves as the foundation for system design, development, testing, and acceptance within the scope of the SWP391 – Software Development Project course.

1.2. Project Scope

AITA-Intelligent is an intelligent programming education support platform, consisting of:

- **Web & Mobile Portal** for students and lecturers (Portal & Auth).
- **Automated code grading system** within isolated Docker environments (Autograding Sandbox).
- **Generative AI module** (GenAI) supporting assignment creation, Clean Code grading, and compilation error explanation.
- **Source code plagiarism detection tool** based on Abstract Syntax Tree (AST) analysis combined with the Winnowing algorithm.
- **Background processing queue** (Redis Queue) and team contribution analysis via Git Analytics.

1.3. Intended Audience

Audience	Purpose
Instructor	Product evaluation and acceptance

Audience	Purpose
Development Team (Dev Team)	Design and development reference
P2P Peer Reviewers	Requirement completeness review
Defense Committee	Final evaluation

1.4. Glossary

Term	Definition
AST	Abstract Syntax Tree – A tree representation of the logical structure of source code
Winnowing	An algorithm for creating digital fingerprints from k-gram sequences for text/code matching
Sandbox	An isolated code execution environment (Docker Container) with limited resources
JWT	JSON Web Token – A stateless authentication standard
SSO	Single Sign-On – One-click login via Google OAuth 2.0
BullMQ	A Redis-based queue library for Node.js
LLM	Large Language Model – e.g., GPT-4o, Gemini
ERD	Entity-Relationship Diagram
SRS	Software Requirements Specification
P2P	Peer-to-Peer – Cross-team evaluation
UAT	User Acceptance Testing
CI/CD	Continuous Integration / Continuous Deployment
LOC	Lines of Code
k-gram	A contiguous subsequence of k elements, used in the Winnowing algorithm
Fingerprint	Digital fingerprint – A set of hash values representing a document/source code

1.5. References

1. Schleimer, S., Wilkerson, D.S., Aiken, A. (2003). "*Winnowing: Local Algorithms for Document Fingerprinting*". ACM SIGMOD 2003.
2. Parr, T. (2010). "*Language Implementation Patterns*". Pragmatic Bookshelf.
3. Sommerville, I. (2016). "*Software Engineering*", 10th Edition. Pearson.
4. Docker Engine API Documentation – https://docs.docker.com/engine/api/
5. OpenAI API Reference – https://platform.openai.com/docs/api-reference

---

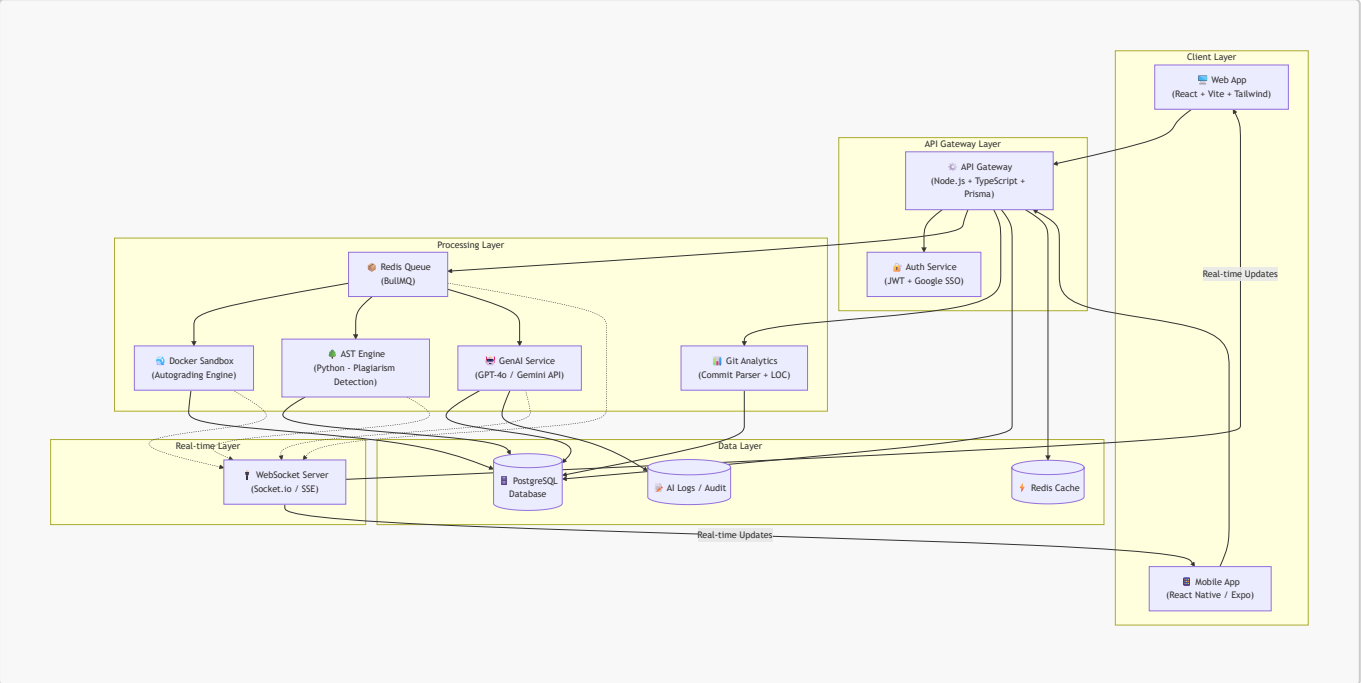
2. System Overview

2.1. Product Context

The AITA-Intelligent system addresses real-world challenges in programming education at universities:

- **Time-consuming manual grading:** Lecturers spend hours grading code for large classes. AITA automates this process using Docker Sandbox.
- **Sophisticated source code fraud:** Students use AI (ChatGPT, Copilot) to write submissions and then disguise them (rename variables, reorder functions). Traditional text comparison tools are ineffective. AITA uses AST + Winnowing for detection.
- **Lack of quality feedback:** Students only see scores without understanding their mistakes. AITA integrates LLM to explain errors in natural language.

2.2. High-Level Architecture

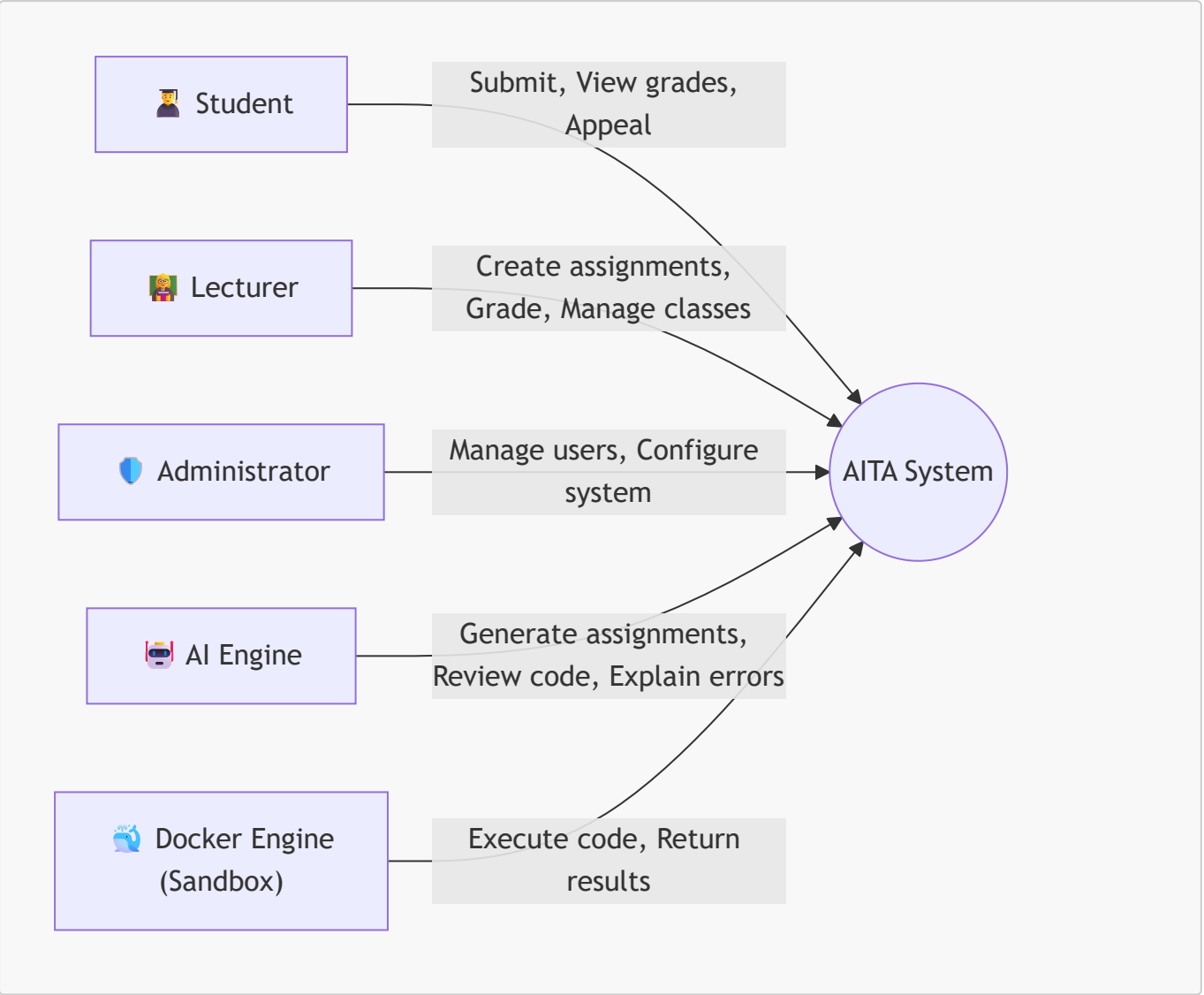


2.3. Core Subsystems

#	Subsystem	Description	Key Technology
1	Portal & Auth	User interface, authentication, authorization	React, Vite, Tailwind, JWT, Google OAuth 2.0
2	Docker Autograding Sandbox	Automated code grading in isolated containers	Docker Engine API, Node.js
3	GenAI Core & Review Hub	Assignment generation, Clean Code grading, error explanation	OpenAI GPT-4o / Gemini API, LangChain
4	AST Plagiarism Detection	Source code plagiarism detection using AST + Winnowing	Python `ast` module, FastAPI
5	Redis Queue & Git Analytics	Background processing queue, team contribution analysis	BullMQ, Redis, Git CLI Parser

3. System Actors

3.1. Actor Diagram



3.2. Detailed Actor Descriptions

Actor	Role	Key Permissions
Student	Students use the system to submit assignments, view grading results, and appeal if they disagree with the outcome	SSO login (@fpt.edu.vn), view assignment list, upload .zip file, view real-time results, submit appeal
Lecturer	Lecturers create classes, design exams, manage grading rubrics, monitor results, and handle complaints	Create/edit/delete classes, import student lists from Excel, create assignments + test cases, request AI-generated assignments/rubrics, view statistical reports, handle appeals, view Git Analytics
Administrator	System administrators manage accounts, configure Docker, and monitor system operations	Manage users, configure Sandbox (RAM, CPU limits), manage API keys (OpenAI/Gemini), view audit logs
AI Engine	Automated system actor – Not a human. Performs content	Generate assignments from lecturer descriptions, generate rubrics, evaluate Clean Code (SOLID,

Actor	Role	Key Permissions
	generation and code quality evaluation tasks	naming, architecture), explain compilation errors in natural language
Docker Engine	Automated system actor. An isolated runtime that executes student-submitted source code	Create/destroy containers, limit resources (512MB RAM, CPU shares), disable external network, capture stdout/stderr, enforce timeout

4. User Stories

4.1. Student Features

ID	User Story	Acceptance Criteria	Priority
US-01	As a **student**, I want to **log in using my FPT Google account** (`@fpt.edu.vn` / `@fe.edu.vn`) so I don't need to create a new account and my identity is verified.	- Only accept FPT domain emails - Redirect to Dashboard after login - JWT stored in HttpOnly Cookie	🔴 High
US-02	As a **student**, I want to **view the list of Assignments** for my enrolled class, so I can track deadlines and submission status.	- Display assignment name, deadline, status (Not submitted / Submitted / Graded) - Sort by nearest deadline	🔴 High
US-03	As a **student**, I want to **submit assignments as a .zip file** containing source code, so the system can automatically grade them for me.	- Only accept .zip files (max 10MB) - File hashed with SHA-256 for integrity verification - System displays "Submission received" immediately	🔴 High
US-04	As a **student**, I want to **view grading results in real-time** (test case scores, Clean Code score, plagiarism check results) as soon as the system finishes processing, without needing to reload the page.	- Real-time updates via WebSocket - Clearly display: Passed/Failed for each test case - Display Clean Code score with AI explanation - Display plagiarism similarity percentage (red warning if > 30%)	🔴 High

ID	User Story	Acceptance Criteria	Priority
US-05	As a **student**, I want to **submit an Appeal** if I believe the automated grading result is inaccurate, so the lecturer can re-evaluate.	- Appeal form with "Reason" field (textarea) - Appeal status: Pending / Accepted / Rejected - Student receives notification when lecturer responds	🟡 Medium

4.2. Lecturer Features

ID	User Story	Acceptance Criteria	Priority
US-06	As a **lecturer**, I want to **create a new class and import student lists from an Excel file**, to save time on manual data entry.	- Upload .xlsx file, system auto-parses columns: Student ID, Full Name, Email - Use DB Transaction to ensure full import or rollback on error - Report successful/failed import counts	🔴 High
US-07	As a **lecturer**, I want to **create Assignments with test cases** (StdIn/StdOut format), so the system can automatically grade student submissions.	- Input title, description, deadline - Add multiple test cases (each with: Input, Expected Output, score weight) - Select programming language for Sandbox (Java, Python, C#)	🔴 High
US-08	As a **lecturer**, I want to **request AI to automatically generate assignments and grading rubrics** from my brief description, to reduce assignment preparation time.	- Lecturer enters a prompt describing the topic (e.g., "Assignment on linked list") - AI returns: complete assignment, sample input/output, grading rubric - Lecturer can edit before saving	🟡 Medium
US-09	As a **lecturer**, I want to **view a Dashboard with overview statistics** (submission rate, score distribution, plagiarism suspects) for each assignment, to quickly assess class performance.	- Pie chart: submission rate - Bar chart: score distribution - Table: Top 10 submission pairs with highest similarity % (AST Plagiarism)	🔴 High
US-10	As a **lecturer**, I want to **handle student appeals** (Accept or Reject with reason), to ensure fairness in grading.	- List of Pending appeals - Review code + original grading results	🟡 Medium

ID	User Story	Acceptance Criteria	Priority
		- Accept (re-grade) or Reject (with reason) button	
US-11	As a **lecturer**, I want to **view Git Analytics reports** for each team member (commits, LOC, PRs), to evaluate individual contributions and detect free-riding.	- Input GitHub Repo URL of the team - Display: Total commits, LOC added/removed, PRs merged, contribution chart over time - Red warning if a member contributes < 5%	🟡 Medium

4.3. System / AI / Docker Features

ID	User Story	Acceptance Criteria	Priority
US-12	As the **system**, when receiving a .zip submission file, I must **enqueue the submission into Redis (BullMQ)** for asynchronous processing, to avoid blocking the main API thread.	- Submission enqueued with metadata: submissionId, studentId, language - Queue has retry mechanism (max 3 retries) on job failure - API immediately returns status 202 Accepted	🔴 High
US-13	As the **Docker Sandbox system**, when receiving a job from Redis Queue, I must **create an isolated container** with limits: 512MB RAM, limited CPU shares, **completely disabled network access**, compile and run student code with test cases, then return results.	- Container self-destructs after completion or timeout (30 seconds) - Prevent: fork bomb, host file reads, disk quota overflow - Return results: Passed/Failed for each test case + stdout/stderr	🔴 High
US-14	As the **AST Engine system**, after Docker finishes grading, I must **convert source code to an AST, remove surface-level disguises** (variable names, comments, function order), **apply Winnowing** to create fingerprints, and **compare against all other submissions** in the same assignment to calculate similarity %.	- Support parsing: Python (`ast` module), C# (Roslyn), Java (ANTLR) - Remove: variable names, function names, comments, whitespace, function order - Winnowing: k-gram size = 25, window size = 40 - Store fingerprints in `ASTFingerprints` table - Create Similarity Matrix: all submission pairs	🔴 High

ID	User Story	Acceptance Criteria	Priority
US-15	As the **AI system (LLM)**, after Docker finishes grading, I must **evaluate source code quality** (Clean Code, SOLID, naming convention, layer architecture) and **explain compilation errors in Vietnamese natural language** so students can understand.	- Use GPT-4o or Gemini API (round-robin key rotation) - Chain-of-Thought + Few-Shot Learning prompts - Return: Clean Code score (0-100), specific comments per file, compilation error explanation if any - Timeout: max 60 seconds/request	🟡 Medium

5. Detailed Use Case Specifications

UC-01: Student Submits Assignment (Submit Assignment)

Attribute	Description
Primary Actor	Student
Secondary Actors	Docker Sandbox, AST Engine, AI Engine
Preconditions	Student is logged in, Assignment deadline has not passed
Main Flow	1. Student selects Assignment from the list 2. Student uploads `.zip` file containing source code 3. System validates file (checks size ≤ 10MB, .zip format) 4. System hashes file with SHA-256 for integrity verification 5. System creates `Submission` record with `PENDING` status 6. System pushes job to Redis Queue (BullMQ) 7. Redis Queue dispatches job to Docker Sandbox 8. Docker Sandbox creates isolated container, compiles and runs test cases 9. Docker returns test results → System updates status to `GRADING` 10. AST Engine parses source code, creates fingerprint, checks for plagiarism 11. AI Engine evaluates Clean Code + explains errors 12. System aggregates final score, updates status to `COMPLETED` 13. Sends real-time results to Student via WebSocket
Alternative Flow	**4a.** File is corrupt or exceeds size limit → Display error, request resubmission **8a.** Student code contains malware (fork bomb) → Docker kills container after 30s timeout → Status `SECURITY_VIOLATION` **8b.** Code fails to compile → Return compilation error + AI error explanation **10a.** Similarity detected > 80% → Flag as `PLAGIARISM_DETECTED`, notify lecturer
Exception Flow	**7a.** Redis Queue is full or down → System retries 3 times, if still failing → Status `QUEUE_ERROR`, notify Admin

Attribute	Description
	11a. OpenAI/Gemini API timeout → Skip Clean Code score, grade based on test cases + AST, mark "AI Review Pending"
Postconditions	`Submission` record is updated with final status and score. Student receives results on the interface.

UC-02: Lecturer Creates Assignment with AI (Create Assignment with AI)

Attribute	Description
Primary Actor	Lecturer
Secondary Actors	AI Engine
Preconditions	Lecturer is logged in, has created at least 1 class
Main Flow	1. Lecturer selects class and clicks "Create New Assignment" 2. Lecturer enters: Title, General Description, Programming Language, Deadline 3. (Optional) Lecturer clicks "Generate with AI": enters a topic description prompt 4. AI Engine returns: Detailed assignment, sample Input/Output, Grading Rubric 5. Lecturer reviews and edits AI-generated content 6. Lecturer adds Test Cases (StdIn → Expected StdOut) manually or from AI 7. Lecturer clicks "Save & Publish" 8. System creates Assignment record, sends notification to all students in the class
Alternative Flow	**3a.** Lecturer doesn't use AI → Manually enters the entire assignment **4a.** AI returns unsuitable results → Lecturer clicks "Regenerate" with a different prompt **6a.** Test cases are invalid (Expected Output is empty) → System warns and requests correction
Postconditions	Assignment is created and displayed in the class assignment list.

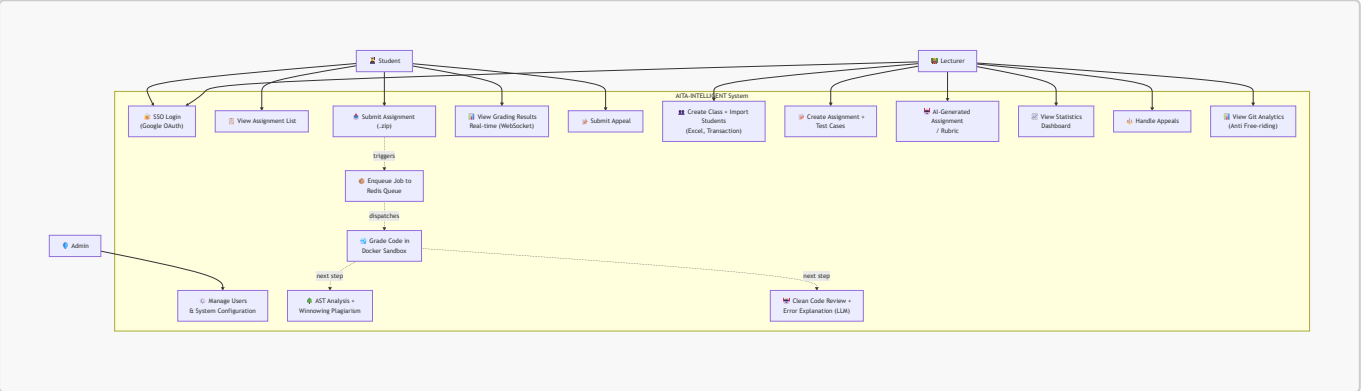
UC-03: Lecturer Imports Student List (Bulk Import Students)

Attribute	Description
Primary Actor	Lecturer
Preconditions	Lecturer has created a class, has a properly formatted Excel file
Main Flow	1. Lecturer selects class, clicks "Import Students" 2. Lecturer uploads `.xlsx` file 3. System parses file, extracts: Student ID, Full Name, Email 4. System displays preview list (for lecturer to verify) 5. Lecturer clicks "Confirm Import" 6. System uses **SQL Transaction** to insert all students 7. Displays results: X students successful, Y duplicate records

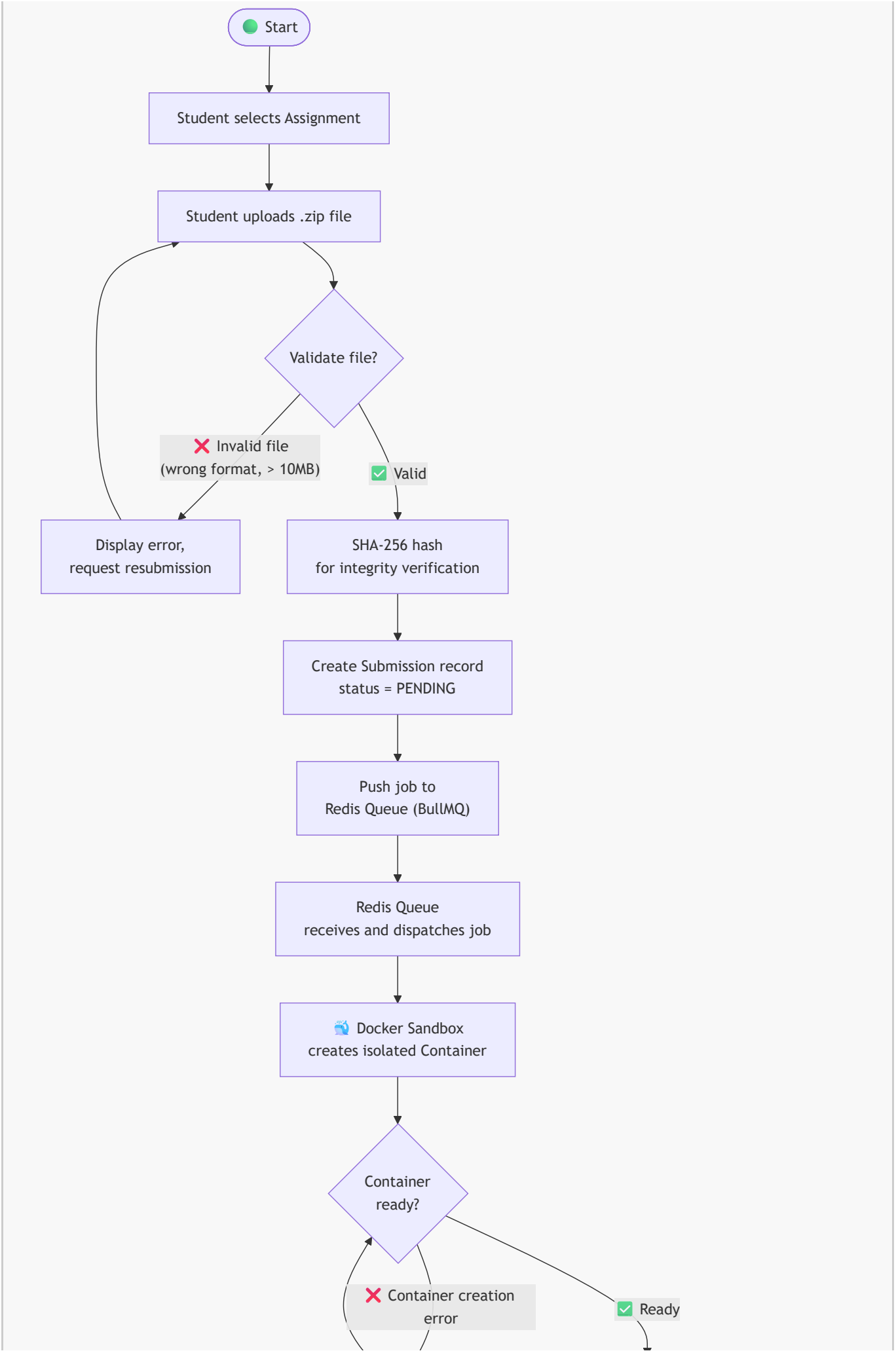
Attribute	Description
Alternative Flow	**3a.** Excel file has wrong format (missing columns) → Display detailed error, request re-upload **6a.** Transaction fails (e.g., duplicate email) → Rollback entirely, report specific duplicate rows
Postconditions	Student list is added to the class. New accounts are created (if not already existing).

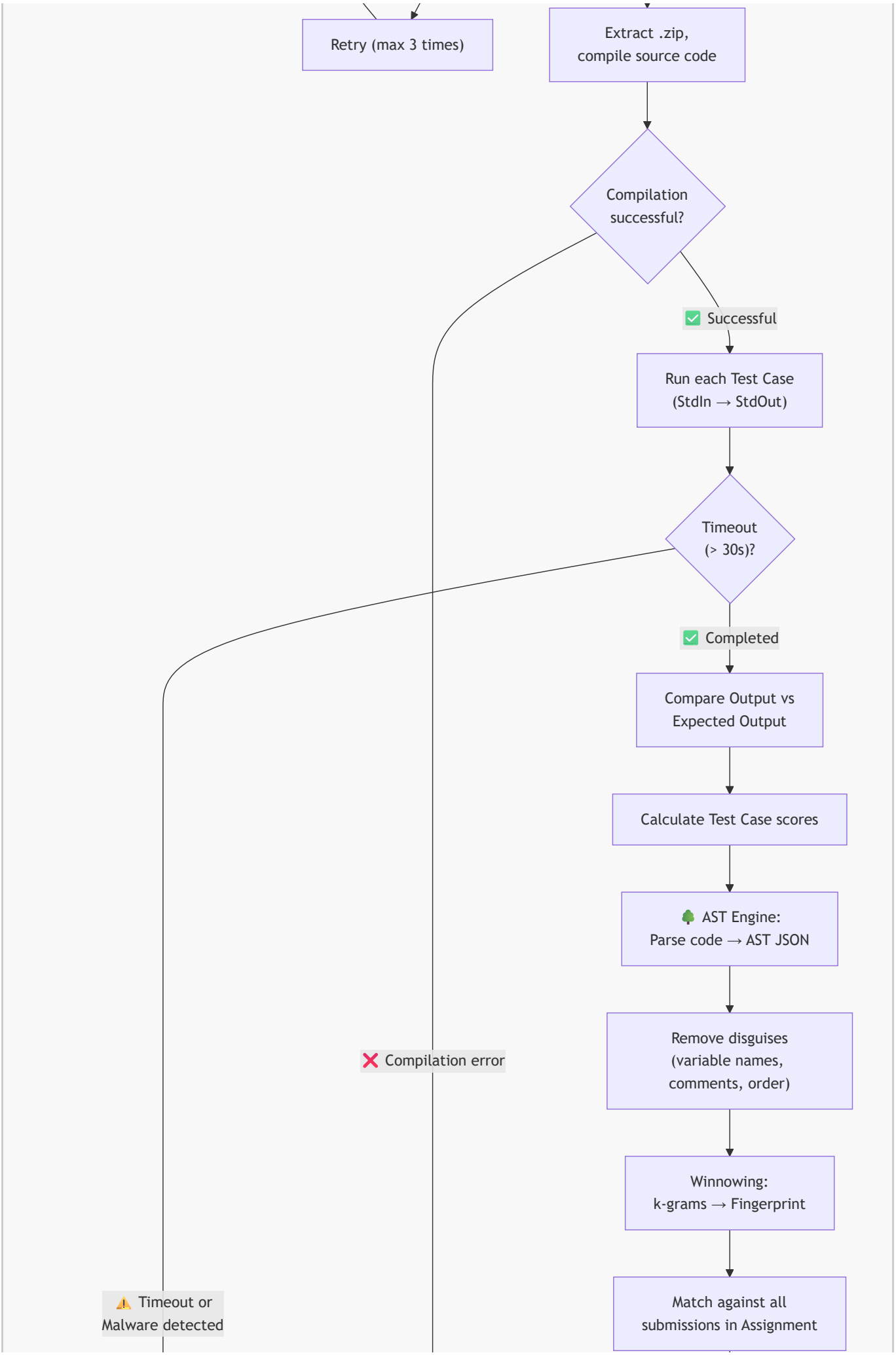
6. UML Diagrams

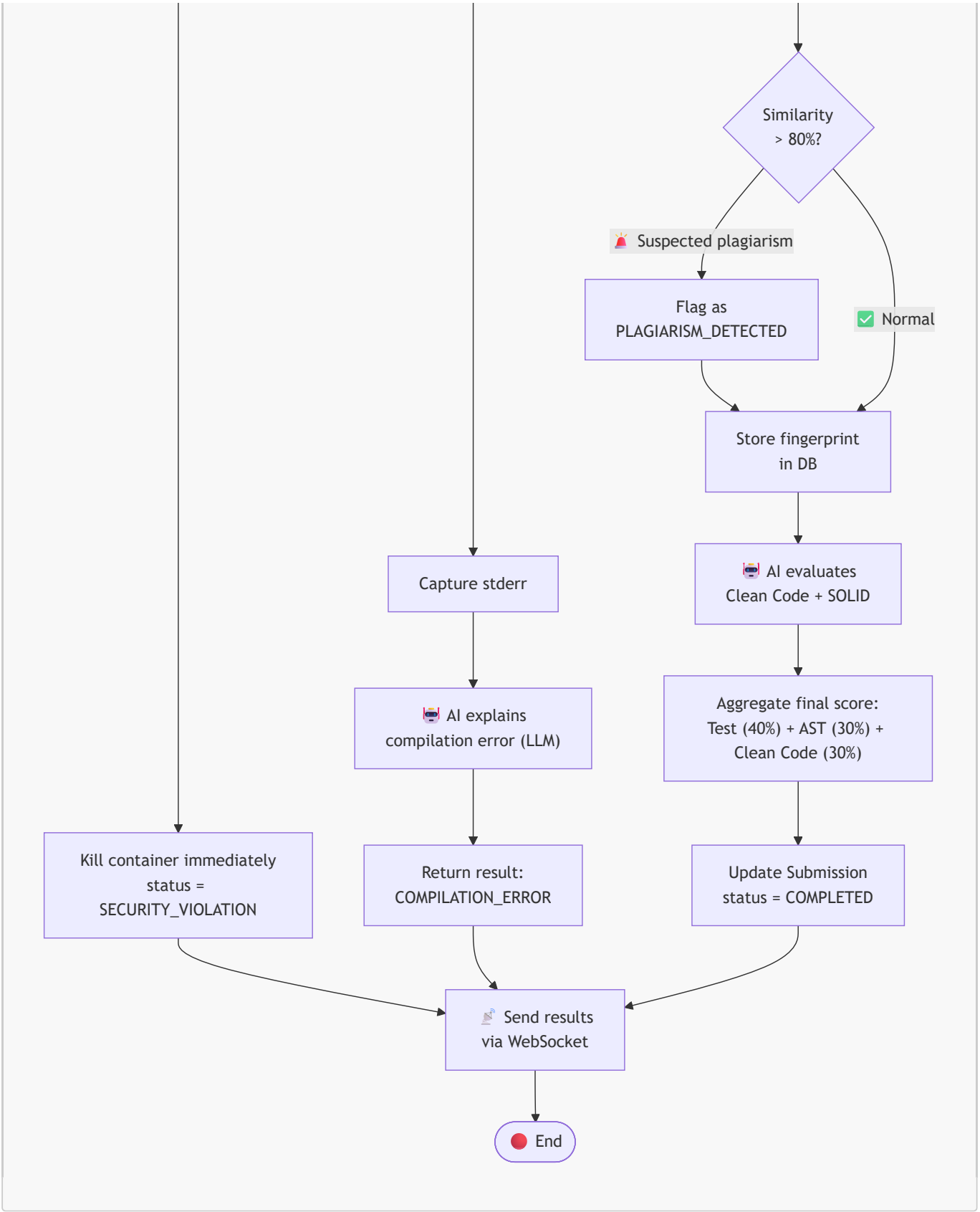
6.1. Use Case Diagram (Overview)



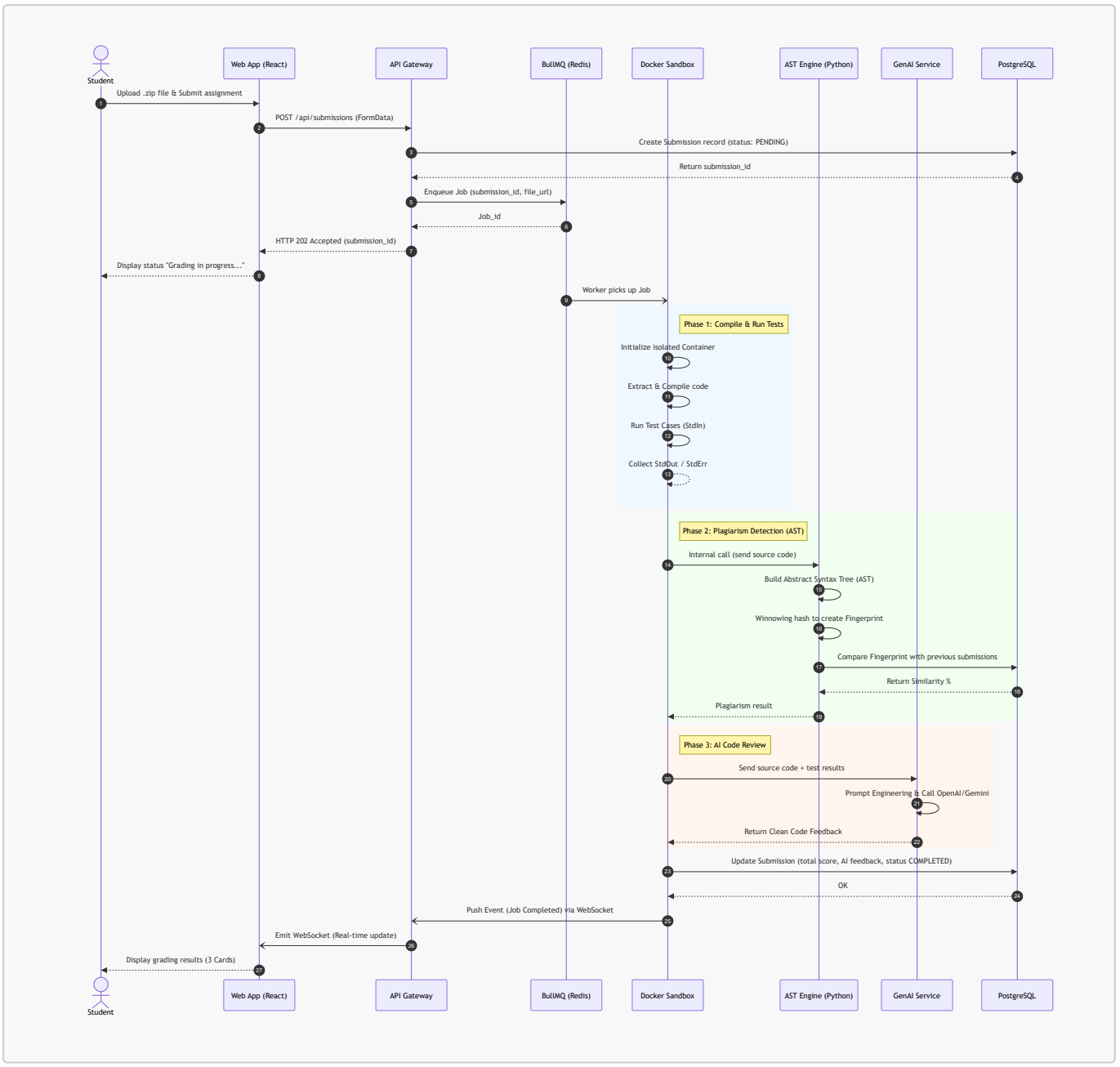
6.2. Activity Diagram – Submission and Automated Grading Flow


Start
Student selects Assignment
Student uploads .zip file
Validate file?
❌ Invalid file
(wrong format, > 10MB)
✅ Valid
Display error,
request resubmission
SHA-256 hash
for integrity verification
Create Submission record
status = PENDING
Push job to
Redis Queue (BullMQ)
Redis Queue
receives and dispatches job
🐳 Docker Sandbox
creates isolated Container
Container
ready?
❌ Container creation
error
✅ Ready


Retry (max 3 times)
Extract .zip,
compile source code
Compilation
successful?
✅ Successful
Run each Test Case
(StdIn → StdOut)
Timeout
(> 30s)?
✅ Completed
Compare Output vs
Expected Output
Calculate Test Case scores
🌳 AST Engine:
Parse code → AST JSON
Remove disguises
(variable names,
comments, order)
Winnowing:
k-grams → Fingerprint
Match against all
submissions in Assignment
❌ Compilation error
⚠️ Timeout or
Malware detected

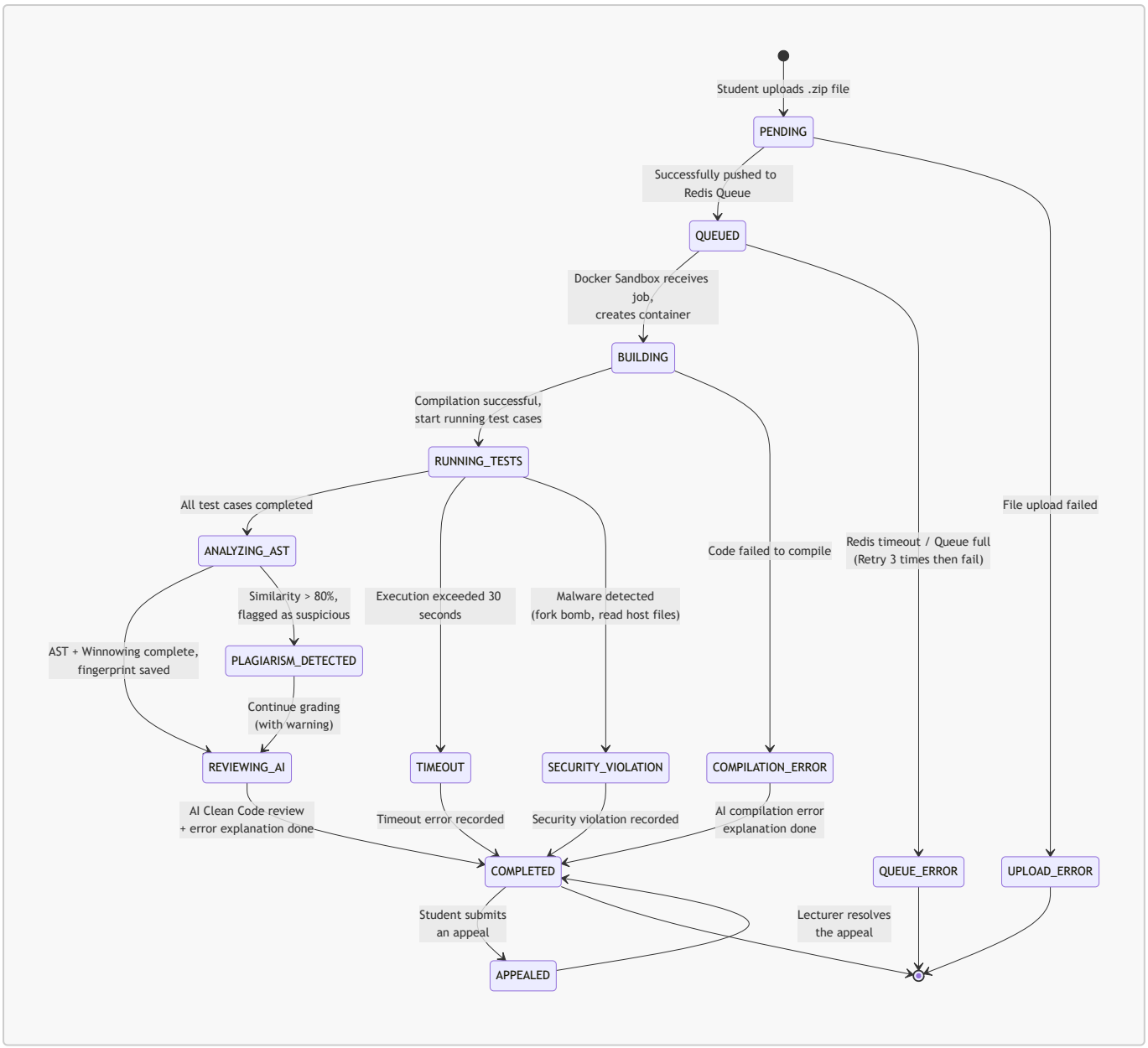


6.3. Sequence Diagram – Submission Grading API Flow

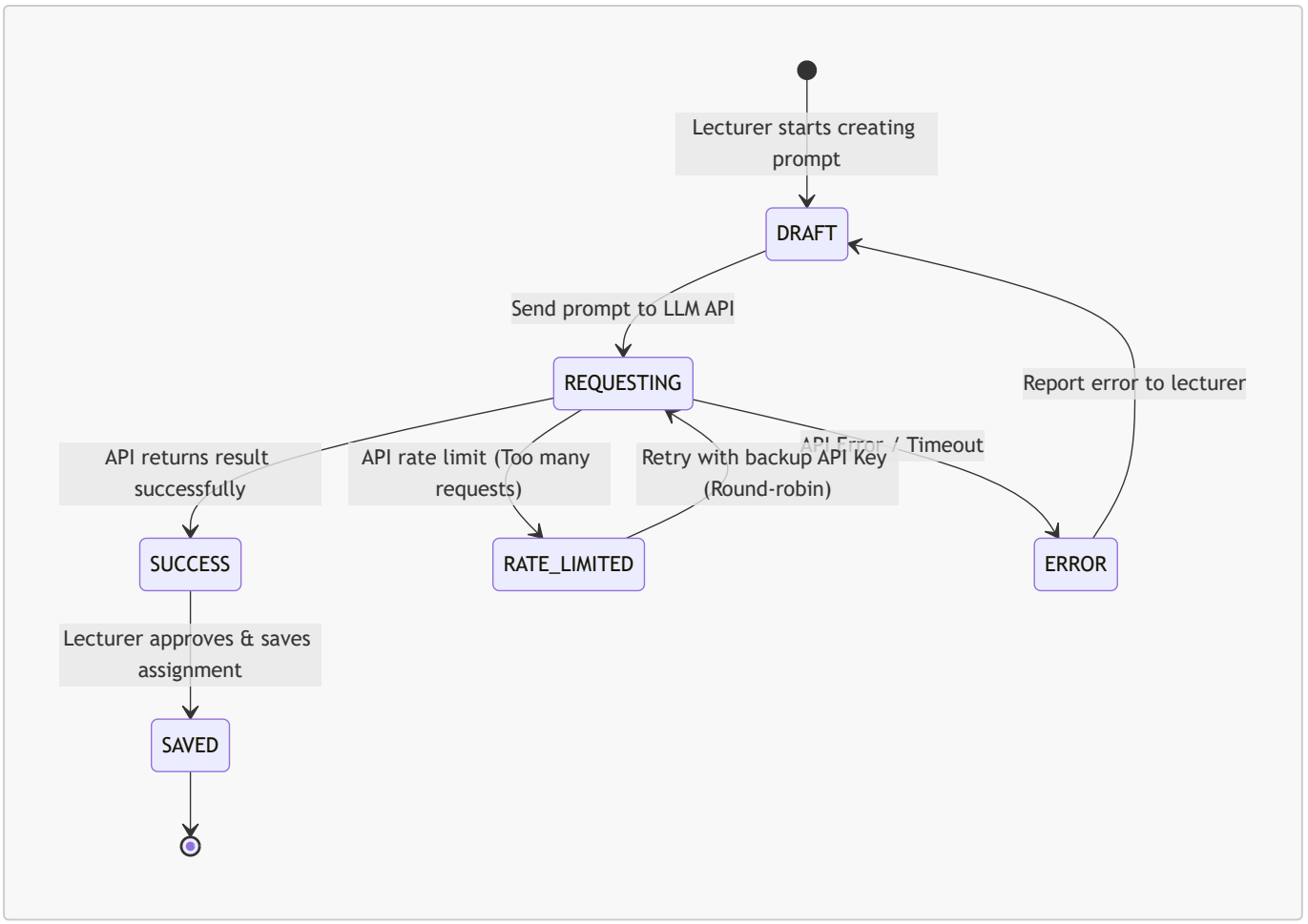


6.4. State Diagrams

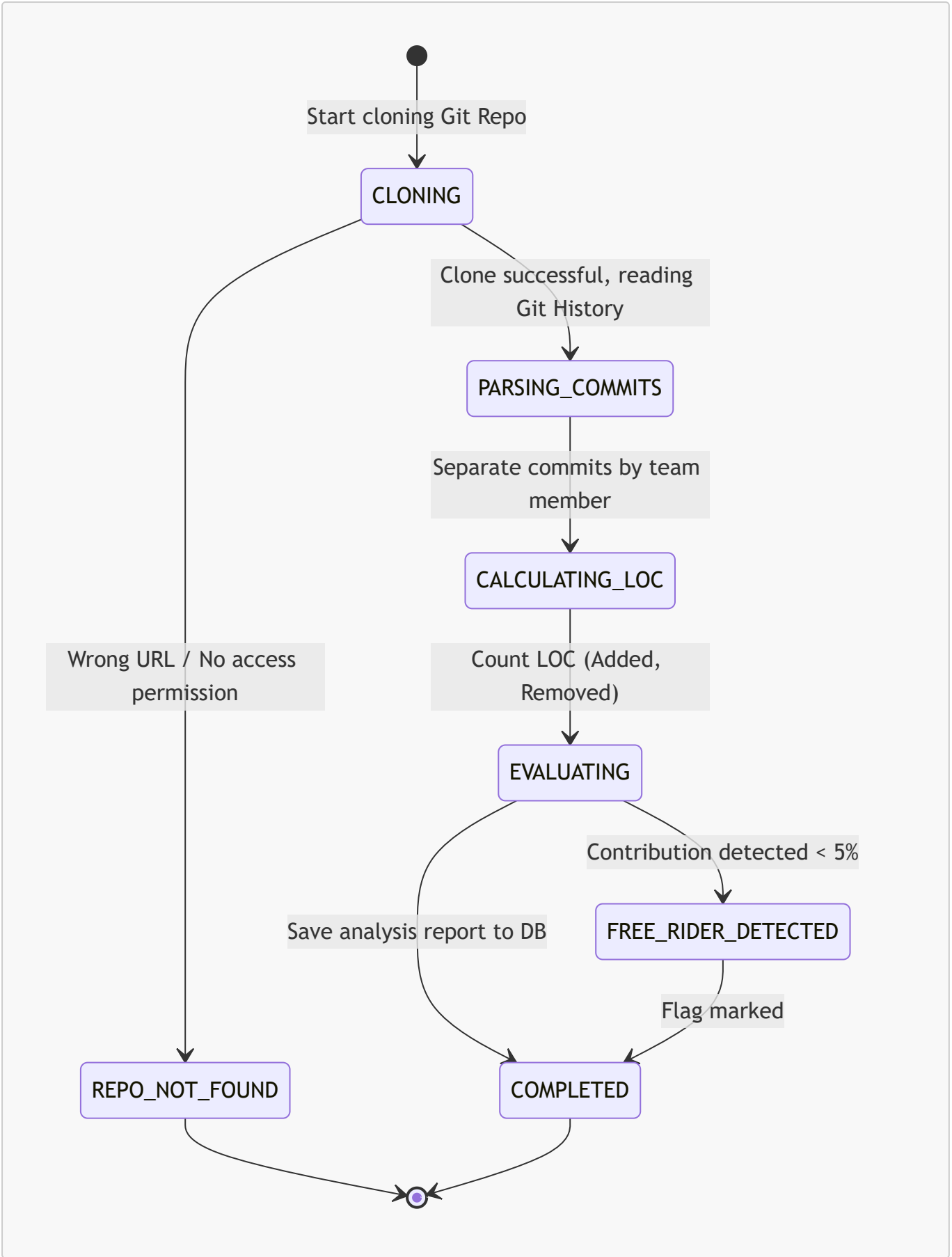
6.4.1. Submission Lifecycle



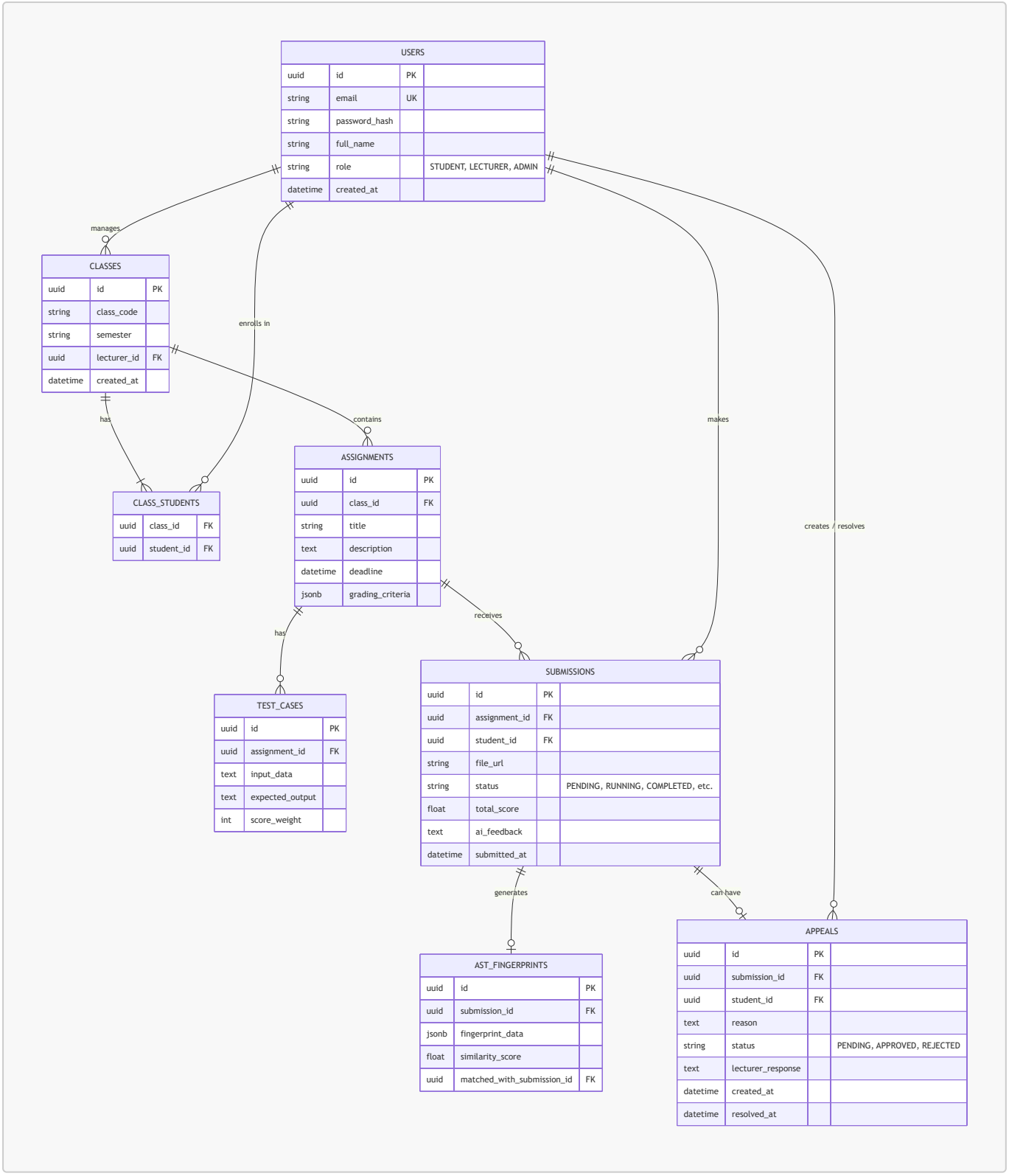
6.4.2. GenAI Request Lifecycle



6.4.3. Git Analytics Process Lifecycle



6.5. ERD (Entity-Relationship Diagram)



6.6. Data Dictionary

Table	Column	Data Type	Constraints	Description
USERS	`id`	UUID	PK	Primary key, user identifier
	`email`	VARCHAR(255)	UNIQUE, NOT NULL	FPT email for login

Table	Column	Data Type	Constraints	Description
	password_hash	VARCHAR(255)		Password (if using traditional login)
	full_name	VARCHAR(100)	NOT NULL	Full name
	role	ENUM	NOT NULL	Role: STUDENT, LECTURER, ADMIN
CLASSES	id	UUID	PK	Primary key, class identifier
	class_code	VARCHAR(50)	NOT NULL	Class code (e.g., SE1801)
	lecturer_id	UUID	FK -> USERS(id)	Lecturer in charge of the class
CLASS_STUDENTS	class_id	UUID	FK -> CLASSES(id)	Foreign key to class
	student_id	UUID	FK -> USERS(id)	Foreign key to student
ASSIGNMENTS	id	UUID	PK	Primary key, assignment identifier
	class_id	UUID	FK -> CLASSES(id)	Foreign key to class
	title	VARCHAR(255)	NOT NULL	Assignment title
	deadline	TIMESTAMP	NOT NULL	Submission deadline
	grading_criteria	JSONB		AI-generated grading criteria
TEST_CASES	id	UUID	PK	Primary key, test case identifier
	assignment_id	UUID	FK -> ASSIGNMENTS(id)	Belongs to which assignment
	input_data	TEXT		Input (StdIn)
	expected_output	TEXT		Expected output (StdOut)
	score_weight	INT	NOT NULL	Score weight

Table	Column	Data Type	Constraints	Description
SUBMISSIONS	`id`	UUID	PK	Primary key, submission identifier
	`student_id`	UUID	FK -> USERS(id)	Submitter
	`file_url`	VARCHAR(255)	NOT NULL	File link .zip (S3/Local)
	`status`	ENUM	NOT NULL	Status (PENDING, QUEUED, COMPLETED...)
	`total_score`	FLOAT		Total score
	`ai_feedback`	TEXT		AI Clean Code feedback
AST_FINGERPRINTS	`id`	UUID	PK	Primary key, AST fingerprint identifier
	`submission_id`	UUID	FK -> SUBMISSIONS(id)	Fingerprint of which submission
	`fingerprint_data`	JSONB	NOT NULL	Winnowing k-gram hash array
	`similarity_score`	FLOAT		Highest similarity %
APPEALS	`id`	UUID	PK	Primary key, appeal identifier
	`submission_id`	UUID	FK -> SUBMISSIONS(id)	Appeal for which submission
	`reason`	TEXT	NOT NULL	Student's appeal reason
	`status`	ENUM	DEFAULT 'PENDING'	Status (PENDING, APPROVED, REJECTED)
	`lecturer_response`	TEXT		Lecturer's response

7. Non-Functional Requirements

7.1. Security

ID	Requirement	Measurement Criteria
NFR-01	Docker Sandbox must completely isolate resources	RAM ≤ 512MB, limited CPU shares, **external network 100% disabled**
NFR-02	Sandbox must defend against common malware	Block: fork bomb, symlink escape, /proc mount, disk exhaustion
NFR-03	JWT authentication must be stored in HttpOnly Cookie	Cookie inaccessible from JavaScript (XSS prevention)
NFR-04	Google SSO must only accept FPT email domains	Whitelist: `@fpt.edu.vn`, `@fe.edu.vn`
NFR-05	API keys (OpenAI, Gemini) must not be hard-coded	Store in environment variables (.env), never commit to Git
NFR-18	**(P2P Defense)** System must log every Sandbox execution in detail	Log includes: `submission_id`, `start_time`, `end_time`, `exit_code`, `resource_usage` (RAM/CPU peak), `security_alert`. Retain for minimum 30 days.

7.2. Performance

ID	Requirement	Measurement Criteria
NFR-06	Average API response time	≤ 200ms for standard APIs (CRUD)
NFR-07	End-to-end grading time per submission	≤ 90 seconds (includes: Docker + AST + AI)
NFR-08	System must handle concurrent load	≥ 100 requests/minute (stress test with k6 or JMeter)
NFR-09	Grading results must be cached	Redis cache for result APIs, TTL = 5 minutes, response ≤ 100ms

7.3. Scalability

ID	Requirement	Measurement Criteria
NFR-10	Redis Queue must support batch grading	Process ≥ 50 concurrent submissions without blocking API gateway
NFR-11	System must support adding new programming languages	Only need to add 1 new Dockerfile to `docker-templates/` directory

7.4. Reliability

ID	Requirement	Measurement Criteria
NFR-12	Bulk student import must use DB Transaction	Import all successfully OR rollback entirely (ACID)

ID	Requirement	Measurement Criteria
NFR-13	Redis Queue must have retry mechanism	Max 3 retries with exponential backoff
NFR-14	Docker containers must self-destruct after completion	No zombie containers left on the server

7.5. Usability

ID	Requirement	Measurement Criteria
NFR-15	Responsive interface on both Web and Mobile	Support: Desktop (≥ 1024px), Tablet (≥ 768px), Mobile (≥ 375px)
NFR-16	Real-time result updates without page reload	Using WebSocket (Socket.IO)
NFR-17	Compilation error explanations in Vietnamese	AI returns explanations in Vietnamese natural language

---

8. System Constraints

8.1. Technology Constraints

- **Frontend Web:** React 18+ with Vite, TailwindCSS
- **Frontend Mobile:** React Native / Expo
- **Backend:** Node.js 20+ with TypeScript, Prisma ORM
- **AST Engine:** Python 3.11+ with FastAPI
- **Database:** PostgreSQL 15+
- **Message Queue:** Redis 7+ with BullMQ
- **Container Runtime:** Docker Engine 24+
- **CI/CD:** GitHub Actions
- **Cloud Deploy:** Azure / AWS / Vercel (team's choice)

8.2. Business Constraints

- The system only serves students and lecturers belonging to FPT University (verified by email domain).
- Each submission is limited to a maximum of 10MB (.zip file).
- Maximum code execution time is 30 seconds per submission.
- OpenAI/Gemini API keys use round-robin rotation to avoid rate limits.

8.3. Project Constraints

- Development timeline: 10 weeks.
- Development team: 4-6 students.
- Code must pass linting (ESLint/Prettier for TypeScript, Flake8 for Python).
- Unit test coverage ≥ 80% on core modules (required from Milestone 3).

---

9. Appendix – AI Validation Logs

> **Instructions:** The development team records all instances of AI usage (ChatGPT, Gemini, Copilot...) during the analysis, design, and development process. Each log includes: date, AI tool, purpose of use, input prompt, and evaluation of results.

Log #1

Attribute	Content
Date	09/07/2026
AI Tool	Gemini 3.1 Pro (High)
Purpose	Analyze Syllabus and create project directory structure
Input Prompt	"I am starting a new project, can you review the file md RBL_Syllabus_AITA_10Weeks.md... sketch the directory tree for this project"
AI Output	AI read and understood the Syllabus file, proposed a Monorepo structure divided into `apps` (api-gateway, web, sandbox, ast) and `docs`.
Team Evaluation	Accepted 100%. Structure is highly accurate for a P2P Review project.
Evidence Screenshot	*[Team inserts AI chat screenshot #1 here]*

Log #2

Attribute	Content
Date	09/07/2026
AI Tool	Gemini 3.1 Pro (High)
Purpose	Generate User Stories list for SRS document
Input Prompt	"Now I and you will work on item #2... create a plan for me to review before approval"
AI Output	AI generated 15 detailed User Stories divided among Student, Lecturer, and System/Admin.
Team Evaluation	Exceeds Rubric requirements (≥12). Content covers all 5 core subsystems.
Evidence Screenshot	*[Team inserts AI chat screenshot #2 here]*

Log #3

Attribute	Content
Date	09/07/2026

Attribute	Content
AI Tool	Gemini 3.1 Pro (High)
Purpose	Generate UML diagram code (Use Case, Activity, State) using Mermaid
Input Prompt	Auto-generated based on task "Complete UML Use Case Diagram, Activity Diagram, State Diagram"
AI Output	AI wrote Markdown code with integrated Mermaid to directly draw submission flow diagrams and Submission lifecycle state diagrams.
Team Evaluation	Mermaid code is accurate, renders well on GitHub, meets submission flow requirements.
Evidence Screenshot	*[Team inserts AI chat screenshot #3 here]*

Log #4

Attribute	Content
Date	09/07/2026
AI Tool	Gemini 3.1 Pro (High) - Image Generation
Purpose	Create UI Mockups and Screen Flow Map design
Input Prompt	"Use AI image generation tool to create sample UI mockup designs... Generate screen flow images and add to project"
AI Output	AI generated 4 interface images (Login, Dashboard, Submission, Results) in Glassmorphism style and 1 User Flow diagram image.
Team Evaluation	Images are very professional, meeting modern "Aesthetics" requirements. Good reference material for Figma design.
Evidence Screenshot	*[Team inserts AI chat screenshot #4 here]*

Log #5

Attribute	Content
Date	09/07/2026
AI Tool	Gemini 3.1 Pro (High)
Purpose	Design Database Schema (ERD) and Data Dictionary
Input Prompt	"There are a few things to improve, please review (Screenshots suggesting missing ERD, Appeals table, Data Dictionary)"
AI Output	AI created an ERD diagram using Mermaid with 8 tables (including Appeals table) and a detailed Data Dictionary for each column.

Attribute	Content
Team Evaluation	Filled the Rubric gap (ERD ≥ 8 entities). Standardized foreign key (FK) constraints.
Evidence Screenshot	*[Team inserts AI chat screenshot #5 here]*

Log #6

Attribute	Content
Date	09/07/2026
AI Tool	Gemini 3.1 Pro (High)
Purpose	Add Non-Functional Requirements (NFR) for P2P Defense
Input Prompt	"There are a few things to improve, please review (Screenshots suggesting missing NFR for Monitoring/Logging for P2P Defense)"
AI Output	AI created NFR-18 requiring detailed logging of every Docker Sandbox execution (exit_code, RAM/CPU) for a minimum of 30 days.
Team Evaluation	This is a critical feature to defend against malware attacks from other teams. Perfect recommendation.
Evidence Screenshot	*[Team inserts AI chat screenshot #6 here]*

End of SRS Document – Version 1.0 Development Team: SWP391 – Group 3